

01 — ML Foundations: NumPy, Pandas & Data Thinking

Time: ~3-4 hours | **Level:** Complete Beginner

What you'll learn:

- Why ML exists and what problems it solves
- NumPy: the engine under every ML library
- Pandas: loading, cleaning, and exploring data
- Matplotlib & Seaborn: making data visual
- Basic statistics that matter for ML

Prerequisites: Basic Python (variables, loops, functions)

1. Why Machine Learning?

Traditional programming:

Rules + Data → Program → Output

Machine Learning:

Data + Output → ML Algorithm → Rules (learned)

Key Insight: Instead of writing rules by hand, we let the computer *discover* rules from examples.

When to use ML:

- Pattern is too complex to code manually (e.g., recognising faces)
- Rules change over time (e.g., spam detection — spammers adapt)
- You have lots of data with known outcomes (e.g., patient records with diagnoses)

When NOT to use ML:

- Simple `if/else` logic works (e.g., "if age > 18, allow access")
- You have very little data (<100 samples usually)
- You need 100% deterministic, auditable results (e.g., tax calculations)

2. NumPy — The Foundation of Everything

NumPy provides **n-dimensional arrays** (called `ndarray`) that are:

- **Fast:** Written in C under the hood (100x faster than Python lists for math)
- **Memory-efficient:** Stores numbers contiguously in memory
- **Vectorised:** Operations apply to entire arrays at once (no loops needed)

Every ML library (PyTorch, TensorFlow, scikit-learn) is built on NumPy arrays.

```
In [ ]: import numpy as np
```

```
# — Creating arrays —————
# From a Python list
a = np.array([1, 2, 3, 4, 5])
print("1D array:", a)
print("Shape:", a.shape)      # (5,) → 5 elements, 1 dimension
print("Data type:", a.dtype)  # int64 by default

# 2D array (matrix) – think of it as rows x columns
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print("\n2D array:\n", b)
print("Shape:", b.shape)      # (2, 3) → 2 rows, 3 columns
```

```
In [ ]: # — Common array creators —————
zeros = np.zeros((3, 4))      # 3x4 matrix of zeros
ones = np.ones((2, 3))        # 2x3 matrix of ones
identity = np.eye(3)          # 3x3 identity matrix
random = np.random.randn(3, 3) # 3x3 matrix of random numbers (normal distribution)
range_arr = np.arange(0, 10, 2) # [0, 2, 4, 6, 8] – like Python range
linspace = np.linspace(0, 1, 5) # 5 evenly spaced numbers between 0 and 1

print("Zeros:\n", zeros)
print("\nRandom (normal distribution):\n", random)
print("\nLinspace:", linspace)
```

Why does shape matter?

In ML, data is always represented as matrices:

Concept	Shape	Example
Single sample	<code>(n_features,)</code>	<code>[age, weight, height]</code>
Dataset	<code>(n_samples, n_features)</code>	1000 patients × 8 features
Image	<code>(height, width, channels)</code>	224×224×3 (RGB)
Batch of images	<code>(batch, H, W, C)</code>	32×224×224×3
Text (tokenised)	<code>(batch, sequence_length)</code>	16 sentences × 512 tokens

```
In [ ]: # — Vectorised operations (the superpower of NumPy) —————
# Instead of looping, operations apply to ALL elements at once

a = np.array([1, 2, 3, 4, 5])

# Without NumPy (slow Python loop):
result_slow = [x * 2 for x in a]

# With NumPy (fast vectorised):
result_fast = a * 2 # Multiplies EVERY element by 2

print("Multiply by 2:", result_fast)
print("Add 10:", a + 10)
print("Square:", a ** 2)
print("Square root:", np.sqrt(a))
print("Sum:", np.sum(a))
print("Mean:", np.mean(a))
print("Std deviation:", np.std(a))
```

```
In [ ]: # — Matrix operations (crucial for ML) —————

# Matrix multiplication: how neural networks compute outputs!
# If X is (samples, features) and W is (features, outputs)
# then X @ W gives (samples, outputs)

X = np.array([[1, 2],
              [3, 4],
              [5, 6]])

W = np.array([[0.5],
              [0.3]])

output = X @ W # Matrix multiply: (3,2) @ (2,1) → (3,1)
print("Input shape:", X.shape)
print("Weights shape:", W.shape)
print("Output shape:", output.shape)
print("Output:\n", output)

# Verify manually: 1*0.5 + 2*0.3 = 0.5 + 0.6 = 1.1 ✓
print("\nManual check (first row): 1*0.5 + 2*0.3 =", 1*0.5 + 2*0.3)
```

💡 Key Insight

This matrix multiplication is literally what a neural network does.

$$\text{output} = \text{input_data} @ \text{weights} + \text{bias}$$

A neural network is just *many layers* of matrix multiplications with non-linear functions in between. That's it. Everything else is optimisation details.

```
In [ ]: # — Broadcasting – NumPy's magic trick —————
# When arrays have different shapes, NumPy "broadcasts" the smaller one
```

```

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]]) # Shape: (2, 3)

bias = np.array([10, 20, 30]) # Shape: (3,)

# Broadcasting: bias is "stretched" to match matrix's shape
result = matrix + bias
print("Matrix + bias (broadcasted):\n", result)
# Row 0: [1+10, 2+20, 3+30] = [11, 22, 33]
# Row 1: [4+10, 5+20, 6+30] = [14, 25, 36]

# This is how bias is added in neural networks!

```

```

In [ ]: # — Indexing & slicing —————
data = np.random.randint(0, 100, size=(5, 4)) # 5 rows, 4 columns
print("Full data:\n", data)

print("\nFirst row:", data[0]) # Row 0
print("First column:", data[:, 0]) # All rows, column 0
print("Rows 1-3, cols 0-2:\n", data[1:4, 0:3]) # Slice

# Boolean indexing (filtering) – very common in data processing
print("\nValues > 50:", data[data > 50]) # All elements > 50
print("Rows where col 0 > 50:\n", data[data[:, 0] > 50]) # Filter rows

```

3. Pandas — Data Loading & Exploration

Pandas is built on NumPy and provides:

- **DataFrame**: A table (like Excel/SQL) with named columns
- **Series**: A single column
- Easy CSV/JSON/SQL reading
- Powerful filtering, grouping, aggregation

Let's create a synthetic mental health dataset to practise with.

```

In [ ]: import pandas as pd

# — Generate synthetic mental health data —————
np.random.seed(42) # For reproducibility
n = 500

df = pd.DataFrame({
    'age': np.random.randint(18, 75, n),
    'gender': np.random.choice(['Male', 'Female', 'Non-binary'], n, p=[0.45,
    'depression_score': np.random.uniform(0, 20, n).round(1),
    'anxiety_score': np.random.uniform(0, 15, n).round(1),
    'sleep_hours': np.random.normal(7, 1.5, n).round(1).clip(2, 12),
    'exercise_days_per_week': np.random.randint(0, 8, n),
    'therapy': np.random.choice(['Yes', 'No'], n, p=[0.35, 0.65]),
})

```

```

# Make depression score correlate with anxiety (realistic)
df['anxiety_score'] = (df['depression_score'] * 0.6 + np.random.normal(0, 3,
# Well-being inversely correlated with depression
df['wellbeing_score'] = (100 - df['depression_score'] * 4 + np.random.normal

print(f"Dataset: {df.shape[0]} rows x {df.shape[1]} columns")
df.head(10)

```

In []: # — Exploring the data —————

```

# Summary statistics – always do this first!
print("=" * 60)
print("SUMMARY STATISTICS")
print("=" * 60)
print(df.describe())

print("\n" + "=" * 60)
print("DATA TYPES")
print("=" * 60)
print(df.dtypes)

print("\n" + "=" * 60)
print("MISSING VALUES")
print("=" * 60)
print(df.isnull().sum())

print("\n" + "=" * 60)
print("CATEGORICAL VALUE COUNTS")
print("=" * 60)
print(df['gender'].value_counts())
print()
print(df['therapy'].value_counts())

```

In []: # — Filtering & selection —————

```

# Select specific columns
scores = df[['depression_score', 'anxiety_score', 'wellbeing_score']]
print("Only score columns:")
print(scores.head())

# Filter rows: patients with high depression AND low well-being
high_risk = df[(df['depression_score'] > 15) & (df['wellbeing_score'] < 40)]
print(f"\nHigh-risk patients: {len(high_risk)} out of {len(df)} ({len(high_r)

# Group by and aggregate
by_gender = df.groupby('gender')[['depression_score', 'anxiety_score']].mean
print("\nMean scores by gender:")
print(by_gender.round(2))

# Group by therapy status
by_therapy = df.groupby('therapy')['wellbeing_score'].agg(['mean', 'std', 'c
print("\nWell-being by therapy status:")
print(by_therapy.round(2))

```

4. Visualisation — Making Data Tell a Story

A picture is worth a thousand numbers. Visualisation helps you:

- Spot distributions (is the data normal? skewed?)
- Find correlations (do two variables move together?)
- Identify outliers (unusual data points)
- Communicate findings to non-technical stakeholders

```
In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid', font_scale=1.1)

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Distribution of depression scores
sns.histplot(df['depression_score'], kde=True, color='coral', ax=axes[0])
axes[0].set_title('Depression Score Distribution')
axes[0].axvline(df['depression_score'].mean(), color='red', linestyle='--', label='mean')
axes[0].legend()

# Distribution of anxiety scores
sns.histplot(df['anxiety_score'], kde=True, color='steelblue', ax=axes[1])
axes[1].set_title('Anxiety Score Distribution')
axes[1].axvline(df['anxiety_score'].mean(), color='blue', linestyle='--', label='mean')
axes[1].legend()

# Distribution of well-being
sns.histplot(df['wellbeing_score'], kde=True, color='seagreen', ax=axes[2])
axes[2].set_title('Well-being Score Distribution')
axes[2].axvline(df['wellbeing_score'].mean(), color='green', linestyle='--', label='mean')
axes[2].legend()

plt.tight_layout()
plt.show()
```

```
In [ ]: # — Correlation: do variables move together? —————

# Correlation matrix – this is your best friend in EDA
numeric_cols = ['age', 'depression_score', 'anxiety_score', 'sleep_hours',
                'exercise_days_per_week', 'wellbeing_score']
corr_matrix = df[numeric_cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, vmin=-1, vmax=1, square=True)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()

print("\n💡 Key observations:")
```

```
print("• Depression & Anxiety: positively correlated (they co-occur)")
print("• Depression & Well-being: negatively correlated (as one rises, other
```

```
In [ ]: # — Scatter plot: the relationship between two variables —
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Depression vs Anxiety (positive correlation)
sns.scatterplot(data=df, x='depression_score', y='anxiety_score',
                hue='therapy', alpha=0.6, ax=axes[0])
axes[0].set_title('Depression vs Anxiety (coloured by therapy status)')

# Depression vs Well-being (negative correlation)
sns.scatterplot(data=df, x='depression_score', y='wellbeing_score',
                hue='gender', alpha=0.6, ax=axes[1])
axes[1].set_title('Depression vs Well-being (coloured by gender)')

plt.tight_layout()
plt.show()
```

5. Statistics That Matter for ML

You don't need a PhD in statistics for ML, but these concepts are essential:

Concept	What It Means	Why It Matters for ML
Mean	Average value	Baseline prediction ("just predict the average")
Standard Deviation	How spread out the data is	Feature scaling
Correlation	Linear relationship between variables	Feature selection
Distribution	Shape of the data (normal, skewed, etc.)	Choosing the right model
Variance	How much values differ from the mean	Bias-variance tradeoff

```
In [ ]: # — Normal Distribution (the bell curve) —
# Most ML algorithms assume (or work best with) normally distributed features

from scipy import stats

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Normal distribution
normal_data = np.random.normal(loc=0, scale=1, size=10000) # mean=0, std=1
axes[0].hist(normal_data, bins=50, density=True, alpha=0.7, color='steelblue')
axes[0].set_title('Normal Distribution\n(mean=0, std=1)')
axes[0].set_xlabel('Value')

# Right-skewed (common in real data: income, house prices)
```

```

skewed_data = np.random.exponential(scale=2, size=10000)
axes[1].hist(skewed_data, bins=50, density=True, alpha=0.7, color='coral')
axes[1].set_title('Right-Skewed Distribution\n(e.g., income)')
axes[1].set_xlabel('Value')

# Bimodal (two groups mixed)
bimodal = np.concatenate([np.random.normal(-2, 0.8, 5000),
                           np.random.normal(3, 0.8, 5000)])
axes[2].hist(bimodal, bins=50, density=True, alpha=0.7, color='seagreen')
axes[2].set_title('Bimodal Distribution\n(two groups mixed)')
axes[2].set_xlabel('Value')

plt.tight_layout()
plt.show()

print("💡 If your data is skewed, consider log-transforming it before feeding it into ML models.")

```

```

In [ ]: # — Feature Scaling: why it matters —————

# Problem: features on different scales confuse many ML algorithms
print("Before scaling:")
print(f" Age range: {df['age'].min()} - {df['age'].max()}")
print(f" Depression score range: {df['depression_score'].min()} - {df['depression_score'].max()}")
print(f" Well-being score range: {df['wellbeing_score'].min()} - {df['wellbeing_score'].max()}")

# Solution 1: Min-Max Scaling (squash to 0-1)
from sklearn.preprocessing import MinMaxScaler, StandardScaler

minmax = MinMaxScaler()
scaled_minmax = minmax.fit_transform(df[['age', 'depression_score', 'wellbeing_score']])

print("\nAfter Min-Max scaling (0 to 1):")
print(f" Age range: {scaled_minmax[:,0].min():.2f} - {scaled_minmax[:,0].max():.2f}")
print(f" Depression score range: {scaled_minmax[:,1].min():.2f} - {scaled_minmax[:,1].max():.2f}")

# Solution 2: Standard Scaling (mean=0, std=1)
standard = StandardScaler()
scaled_std = standard.fit_transform(df[['age', 'depression_score', 'wellbeing_score']])

print("\nAfter Standard scaling (mean≈0, std≈1):")
print(f" Age mean: {scaled_std[:,0].mean():.4f}, std: {scaled_std[:,0].std():.4f}")
print(f" Depression mean: {scaled_std[:,1].mean():.4f}, std: {scaled_std[:,1].std():.4f}")

print("\n💡 Rule of thumb: Use StandardScaler for most ML models, MinMaxScaler for image data.")

```

6. Feature Engineering — Creating Better Inputs

Raw data is rarely the best input for ML. **Feature engineering** is the art of creating new, more informative features from existing ones. This is often the difference between a mediocre and a great model.

```

In [ ]: # — Creating new features from existing ones —————

```

```

df_engineered = df.copy()

# 1. Binning: convert continuous age to categories
df_engineered['age_group'] = pd.cut(df['age'],
                                    bins=[0, 25, 40, 60, 100],
                                    labels=['Young', 'Adult', 'Middle-aged', 'Senior'])

# 2. Interaction feature: combined distress score
df_engineered['distress_index'] = (df['depression_score'] + df['anxiety_score'])

# 3. Binary flag: is the person high-risk?
df_engineered['high_risk'] = ((df['depression_score'] > 12) |
                              (df['anxiety_score'] > 10)).astype(int)

# 4. Ratio feature: exercise relative to distress
df_engineered['exercise_to_distress'] = (
    df['exercise_days_per_week'] / (df_engineered['distress_index'] + 1) # Avoid division by zero
).round(3)

# 5. One-hot encoding: convert categories to numbers
gender_dummies = pd.get_dummies(df['gender'], prefix='gender', drop_first=True)
therapy_dummy = (df['therapy'] == 'Yes').astype(int).rename('has_therapy')

print("New features:")
print(df_engineered[['age', 'age_group', 'distress_index', 'high_risk',
                    'exercise_to_distress']].head(10))

print("\nOne-hot encoded gender:")
print(gender_dummies.head())

print(f"\n💡 High-risk patients: {df_engineered['high_risk'].sum()} / {len(df_engineered)} ({df_engineered['high_risk'].mean()*100:.1f}%)")

```

Exercises

Try these on your own before moving to the next notebook:

1. **NumPy**: Create a 100×5 matrix of random numbers. Compute the mean and std of each column.
2. **Pandas**: Find the average depression score for each age group (Young, Adult, Middle-aged, Senior).
3. **Visualisation**: Create a box plot showing well-being score by therapy status.
4. **Feature Engineering**: Create a "sleep_quality" feature that's 1 if sleep_hours is between 6-9, else 0.

Key Takeaways

1. **NumPy** is the engine — arrays, matrix math, vectorised operations
2. **Pandas** is for data exploration — loading, filtering, aggregation

3. **Always visualise first** — distributions, correlations, outliers
4. **Feature scaling** is critical — models are confused by different scales
5. **Feature engineering** is often more impactful than model choice

Next: [02 — Classical Machine Learning](#) — where we train our first models!